UPPSALA
UNIVERSITET

Numerical Linear Algebra · 1TD452

---

# Signal and Image Processing

*Mini-project 2: Iterative methods*

---

Pontus Ahlberg
Ivar Hammarberg
Isac Persson

14 October 2026

# Contents

1 Introduction ........ 1

2 Method ........ 1

- 2.1 Images as vectors ........ 1
- 2.2 Least squares ........ 1
- 2.3 Regularization ........ 1
- 2.4 The normal equations ........ 2
- 2.5 Symmetric positive definite matrix ........ 2
- 2.6 Iterative Krylov subspace methods ........ 3
- 2.7 Conjugate Gradient ........ 3
- 2.8 MINRES ........ 4
- 2.9 LSQR ........ 4
- 2.10 Blur matrix $A$ ........ 5
- 2.11 The regularization matrix $L$ ........ 6
- 2.12 Sparsity pattern of $A$ ........ 6
- 2.13 Sparsity pattern of $L$ ........ 7
- 2.14 Influence of $\beta$ and $\varepsilon$ ........ 7

3 Results ........ 8

- 3.1 Influence of the regularization parameter $\lambda$ on CG ........ 8
- 3.2 Influence of the stopping tolerance on CG ........ 8
- 3.3 The maximum number of iterations for CG ........ 9
- 3.4 Comparison of CG and MINRES ........ 9
- 3.5 Why CG is expected to outperform MINRES ........ 10
- 3.6 Preconditioned CG ........ 10

4 Summary ........ 11

References ........ 12

# 1 Introduction

In this project we study iterative methods for solving linear systems. We apply these methods to to recover a sharp image from a blurred one. The task boils down to a very large least squares problem, and with a dense matrix of that size the direct methods introduced earlier become too expensive. We therefore make the matrix artificially sparse and use iterative methods that are built for sparse problems. The deblurring problem is, however, ill-posed, so small amounts of noise can ruin the solution, and we show how regularization turns it into a solvable one.

# 2 Method

Direct methods like GSVD need too much storage and computation for large image problems. We instead use iterative methods, which refine the solution step by step using mainly matrix-vector products and a few stored vectors.

## 2.1 Images as vectors

A grayscale image is a matrix $X \in \mathbb{R}^{n_1 \times n_2}$ of pixel brightnesses from 0 (black) to 1 (white). Stacking its columns gives

$$x_{\text{true}} = \text{vec}(X) \in \mathbb{R}^n, \quad n = n_1 n_2$$

We resize the image to $300 \times 300$ pixels, so $n = 90000$. The goal is to recover the sharp image $x_{\text{true}}$ from the blurred and noisy observation $b = Ax_{\text{true}} + e$ where $A$ is the blurring matrix and $e$ is the noise.

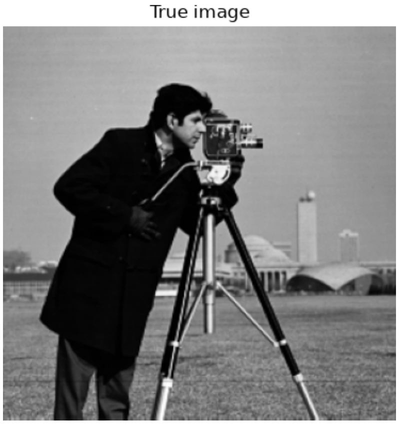

Figure 1: Original $X_{\text{true}}$ image

## 2.2 Least squares

Only $b$ is known, not $x_{\text{true}}$ or $e$. Without noise we could solve $Ax = b$, but noise makes an exact solution unreliable, so we solve

$$\min_x \|Ax - b\|_2^2$$

which minimizes the residual $Ax - b$, the difference between the predicted blurred image and $b$. Least squares alone also fits the noise in $b$, which motivates regularization.

## 2.3 Regularization

To suppress noise, we penalize non-smooth reconstructions. $L$ takes differences between neighbouring pixels, so $\|Lx\|_2^2$ is large when neighbours differ strongly. We minimize

$$f(x) = \underbrace{\|Ax - b\|_2^2}_{\text{fit the observed data}} + \underbrace{\lambda^2 \|Lx\|_2^2}_{\text{encourage smoothness}} \tag{1}$$

where $\lambda > 0$ balances the two terms: too small leaves noise, too large loses details and edges.

## 2.4 The normal equations

We write out $f$ and set its gradient to zero:

$$\begin{aligned} f(x) &= \|Ax-b\|_2^2 + \lambda^2\|Lx\|_2^2 = (Ax-b)^T(Ax-b) + \lambda^2(Lx)^T(Lx) \\ &= x^T(A^TA + \lambda^2L^TL)x - 2x^TA^Tb + b^Tb \\ \Rightarrow \nabla f(x) &= 2(A^TA + \lambda^2L^TL)x - 2A^Tb = 0 \end{aligned} \tag{2}$$

This gives the linear system

$$Mx = c, \quad M = A^TA + \lambda^2L^TL, \quad c = A^Tb \tag{3}$$

This is the system we solve in the rest of the report.

## 2.5 Symmetric positive definite matrix

Let $A \in \mathbb{R}^{n\times n}$ and $L \in \mathbb{R}^{p\times n}$. Assume that $\lambda \neq 0$ and

$$\operatorname{rank}\left(\begin{bmatrix} A \\ L \end{bmatrix}\right) = n$$

Using $M = A^TA + \lambda^2L^TL$ we want to prove that $M$ is symmetric positive definite. A matrix is symmetric if $M^T = M$. Taking the transpose gives

$$\begin{aligned} M^T &= (A^TA + \lambda^2L^TL)^T = (A^TA)^T + \lambda^2(L^TL)^T \\ &= A^TA + \lambda^2L^TL \qquad = M \end{aligned}$$

Therefore, $M$ is symmetric. A symmetric matrix is positive definite if

$$z^TMz > 0 \quad \text{for all } z \neq 0$$

$$\begin{aligned} \Rightarrow z^TMz &= z^T(A^TA + \lambda^2L^TL)z \\ &= z^TA^TAz + \lambda^2z^TL^TLz \\ &= \|Az\|_2^2 + \lambda^2\|Lz\|_2^2 \end{aligned}$$

Since squared Euclidean norms are always non-negative and $\lambda \neq 0$ implies $\lambda^2 > 0$ we get:

$$z^TMz = \|Az\|_2^2 + \lambda^2\|Lz\|_2^2 \geq 0$$

To prove positive definiteness, we must show that the inequality is strict whenever $z \neq 0$. The assumption $\operatorname{rank}\left(\begin{bmatrix} A \\ L \end{bmatrix}\right) = n$ means that the matrix obtained by stacking $A$ and $L$ has full column rank. Therefore, its null space contains only the zero vector:

$$\begin{bmatrix} A \\ L \end{bmatrix} z = 0 \quad \text{Only if} \quad z = 0$$

$$\Rightarrow Az = 0 \text{ and } Lz = 0 \quad \text{Only if} \quad z = 0$$

Now let $z \neq 0$. It follows that $Az$ and $Lz$ cannot both be zero at the same time. In other words at least on is greater than zero. Which means $M$ is symmetric positive definite.

## 2.6 Iterative Krylov subspace methods

An iterative method starts from a guess $x_0$ (we use $x_0 = 0$, a black image) and improves it step by step, $x_1, x_2, \ldots$. To see how close $x_k$ is to solving $Mx = c$ we use the residual.

$$r_k = c - Mx_k$$

which is zero exactly at the solution. We stop when the relative residual $\|r_k\|_2 / \|c\|_2$ is below a tolerance or after a maximum number of iterations.

In the first step the only direction we know is the residual $r_0$, so $x_1 = x_0 + \alpha_0 r_0$ for some number $\alpha_0$. In Krylov subspace methods a new direction is always found for each iteration, and the step length in that direction is chosen so $x_k$ is the best possible approximation that the method can find in *k* steps. The subspace is built from $r_0$ and its products with $M$: first $Mr_0$, then $M(Mr_0) = M^2 r_0$, and so on. $x_k - x_0$ is therefore a linear combination of

$$\mathcal{K}_k(M, r_0) = \operatorname{span}\{r_0, Mr_0, \ldots, M^{k-1} r_0\}$$

The two methods used in this project are Conjugate Gradient (CG) and Minimum Residual (MINRES).

## 2.7 Conjugate Gradient

Since $M$ is SPD, solving $Mx = c$ is the same as minimizing

$$\phi(x) = \frac{1}{2} x^T M x - c^T x$$

We can use the update rule

$$x_{k+1} = x_k + \alpha_k p_k$$

where $p_k$ is the search direction and $\alpha_k$ is the step length. We choose $p_k$ to be $M$-orthogonal to all previous search directions. $p_0$ is chosen to be the initial residual $r_0 = c - Mx_0$. The search direction is updated according to

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

where

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$$

and

$$\alpha_k = \frac{r_k^T r_k}{p_k^T M p_k}$$

When M is SPD, CG converges in at most n iterations, and the error decreases monotonically. The convergence rate depends on the condition number of M, which is improved by regularization.

In this project, CG is implemented in Python, adapted from the algorithm in [1].

## 2.8 MINRES

MINRES (minimal residual) solves $Mx = c$ for symmetric $M$, which unlike in CG does not need to be positive definite. In iteration $k$ it chooses

$$x_k = \underset{x \in x_0 + \mathcal{K}_k(M, r_0)}{\arg\min} \|c - Mx\|_2$$

the iterate with the smallest residual in the Krylov subspace

Since $M$ is symmetric, the Lanczos process reduces this to a small least squares problem with a tridiagonal matrix. This problem is solved with Givens rotations, which gives the short update

$$x_{k+1} = x_k + \tau_k p_k$$

Like CG, MINRES needs one matrix-vector product per iteration and a fixed number of stored vectors. CG minimizes the error in the $M$-norm, MINRES the residual in the 2-norm.

## 2.9 LSQR

There are other iterative methods that can be used to solve the problem. LSQR is a third option, it is not implemented or tested in this project. This section can be skipped without missing anything in the results. LSQR [2], [3] is an iterative method for least squares problems $\min_x \left\|\hat{A}x - \hat{b}\right\|_2$. Unlike CG and MINRES it does not use the normal equations, but works on the least squares problem directly.

The regularized problem Equation 1 becomes an ordinary least squares problem if we stack $A$ on top of $\lambda L$:

$$\left\| \begin{bmatrix} A \\ \lambda L \end{bmatrix} x - \begin{bmatrix} b \\ 0 \end{bmatrix} \right\|_2^2 = \|Ax - b\|_2^2 + \lambda^2 \|Lx\|_2^2$$

So we apply LSQR with $\hat{A} = \begin{bmatrix} A \\ \lambda L \end{bmatrix}$ and $\hat{b} = \begin{bmatrix} b \\ 0 \end{bmatrix}$. In each iteration LSQR only multiplies a vector by $\hat{A}$ and by $\hat{A}^T$:

$$\hat{A}v = \begin{bmatrix} Av \\ \lambda Lv \end{bmatrix}, \quad \hat{A}^T \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = A^T u_1 + \lambda L^T u_2$$

These are sparse matrix-vector products so no matrix-matrix product is needed.

LSQR and CG solve the same problem, since $M = \hat{A}^T \hat{A}$ and $c = \hat{A}^T \hat{b}$, and in exact arithmetic they give the same iterates. Both use four sparse products per iteration and store only a few vectors. The difference is the rounding errors. CG works with $M$, and $\kappa(M) = \kappa\left(\hat{A}\right)^2$, so the errors are amplified more. LSQR works with $\hat{A}$, which makes it more accurate for ill-conditioned problems, that is for small $\lambda$. The disadvantage is that LSQR is more complicated and does slightly more work per iteration, so for a well-conditioned problem CG is just as good and simpler [2].

## 2.10 Blur matrix $A$

To blur the image, the vector $x_{\text{true}}$ is multiplied by a blurring matrix $A$. $A$ comes from the continuous blur model (1.8)

$$b(s) = \int K(s,t)x(t)\,\mathrm{d}t + e(s)$$

where $b(s)$ is the blurred image and $e(s)$ is the noise, which in this case is Gaussian. $K(s,t)$ is called the *point spread function* and determines how much each pixel is affected by every other pixel in the image. We use the Gaussian kernel (1.9) with $B = \beta I$, where $\beta$ controls how strongly pixels far away contribute:

$$K(s,t) = ce^{-\beta\|s-t\|_2^2}$$

To make sure that $K(s,t)$ does not change the overall brightness, $c$ is chosen as $c = \beta/\pi$.

Since the image is discrete, the integral is approximated by a sum, where $h$ is the width of each pixel:

$$(Ax)_i = \sum_j h^2 K(s_i, t_j) x_j$$

Finally, since $\|s-t\|_2^2 = (s_1 - t_1)^2 + (s_2 - t_2)^2$, the Gaussian kernel factors into one kernel for the $x$-direction and one for the $y$-direction. We can therefore build one-dimensional blur matrices $A_1$ and $A_2$ for each direction, and the Kronecker product gives $A$ for the whole image:

$$A = A_2 \otimes A_1$$

The order of the factors follows from $x = \operatorname{vec}(X)$ stacking the columns of $X$: $A_1 \in \mathbb{R}^{n_1 \times n_1}$ blurs every column (vertical direction) and $A_2 \in \mathbb{R}^{n_2 \times n_2}$ every row (horizontal direction), so $Ax = \operatorname{vec}(A_1 X A_2^T)$.

$A$ is now a dense $90\,000 \times 90\,000$ matrix, which requires 64.8 GB of memory. This is unreasonably large, and pixels far apart barely affect each other anyway. So instead of storing $A$ as a dense matrix, entries below a threshold ($\varepsilon$) can be set to zero and the matrix can be stored in CSR (compressed sparse row) format.

## 2.11 The regularization matrix $L$

To penalize images that are not smooth, we need a matrix $L$ that measures how much neighbouring pixels differ. We start in one dimension, where the first-order difference matrix

$$L_1 = \begin{bmatrix} 1 & -1 & & & \\ & 1 & -1 & & \\ & & \ddots & \ddots & \\ & & & 1 & -1 \end{bmatrix} \in \mathbb{R}^{(n_1-1)\times n_1}, \quad (L_1 v)_i = v_i - v_{i+1}$$

computes the difference between each pair of neighbouring entries of a vector $v$.

Since $x_{\text{true}}$ stacks the columns of $X$, applying $L_1$ to every column gives all vertical differences, likewise $L_2$ applied to every row gives all horizontal differences. Adding them together gives L:

$$L = L_G = \begin{bmatrix} I_{n_2} \otimes L_1 \\ L_2 \otimes I_{n_1} \end{bmatrix} \in \mathbb{R}^{179\,400\times 90\,000}$$

Given that $Lx = 0$ is only attained when all pixels have the same value, and $Ax = 0$ when we have the original image exactly we can conclude that rank $\begin{bmatrix} A \\ L \end{bmatrix} = n$

## 2.12 Sparsity pattern of $A$

To see how much we gain from storing $A$ as a sparse matrix, we look at its sparsity pattern, that is, where its nonzero entries are (Figure 2). With $\beta = 10\,000$ and the threshold $\varepsilon = 10^{-4}$, $A$ has $4.36 \cdot 10^6$ nonzeros, which is only $0.0539\%$ of all entries. Stored in CSR format, $A$ therefore needs 52.7 MB instead of 64.8 GB as a dense matrix.

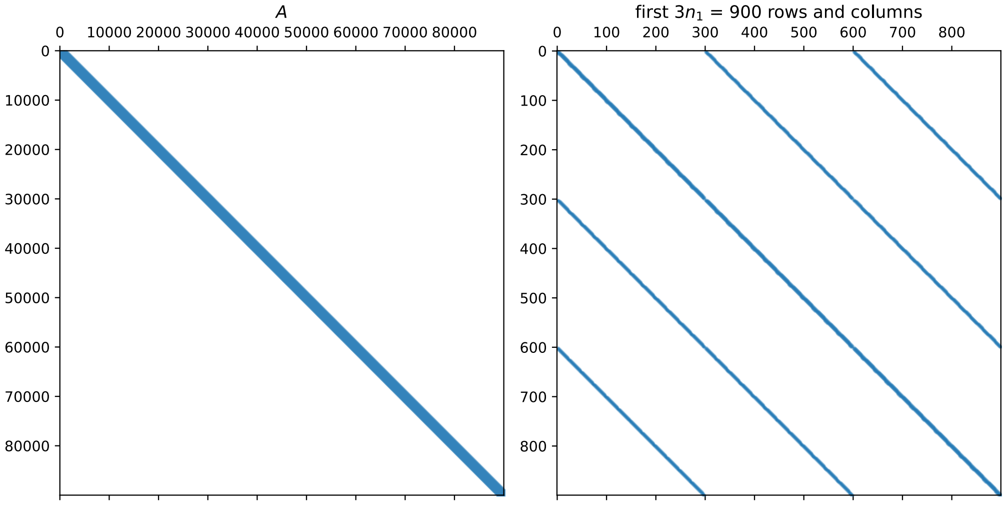

Figure 2: Sparsity pattern of $A$ ($\beta = 10\,000$, $\varepsilon = 10^{-4}$). Left: all of $A$. Right: the first $3n_1$ rows and columns, showing the banded blocks of $A_2 \otimes A_1$.

## 2.13 Sparsity pattern of $L$

$L$ is even sparser than $A$ (Figure 3). Since every row has exactly two nonzeros, $L$ has 358 800 nonzeros, which is only $0.0022\%$ of all entries. In CSR format, $L$ needs 5.02 MB instead of 129 GB as a dense matrix.

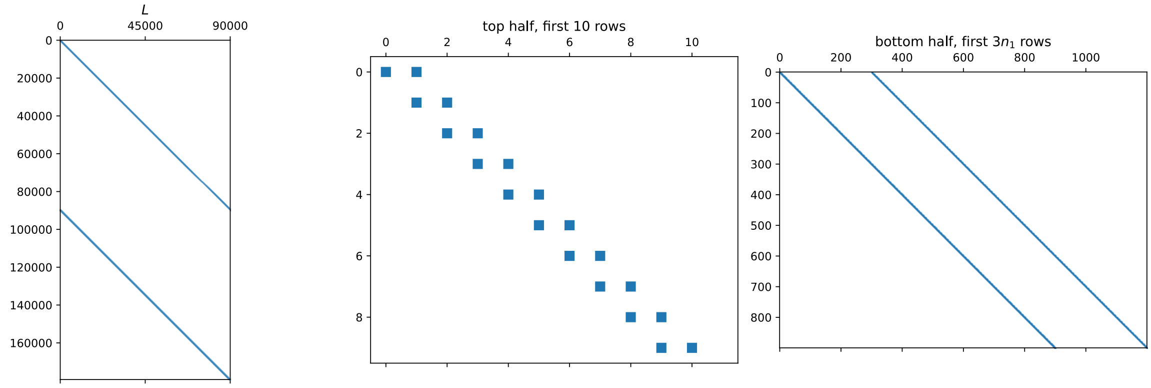

Figure 3: Sparsity pattern of $L$. Left: all of $L$. Middle: the first rows of the top half. Right: the first $3n_1$ rows of the bottom half.

## 2.14 Influence of $\beta$ and $\varepsilon$

To see how the choice of parameters affects the sparsity, we vary $\beta$ and $\varepsilon$ and count the nonzeros of $A$ (Table 1 and Figure 4). Since $\beta$ controls how strongly pixels far away contribute, a larger $\beta$ gives a narrower Gaussian. Fewer entries are then above $\varepsilon$, so $A$ becomes sparser.

A larger $\varepsilon$ also makes $A$ sparser, since more entries are set to zero. The price is that $A$ becomes a worse approximation of the real blur, because more of the Gaussian is cut away.

| $\beta$ | $\varepsilon$ | $\mathrm{nnz}(A)$ | nonzero % |
|---|---|---|---|
| 500 | $10^{-3}$ | $2.43 \cdot 10^7$ | 0.300 |
| 500 | $10^{-4}$ | $5.13 \cdot 10^7$ | 0.633 |
| 500 | $10^{-5}$ | $7.99 \cdot 10^7$ | 0.987 |
| 1 000 | $10^{-3}$ | $1.63 \cdot 10^7$ | 0.201 |
| 1 000 | $10^{-4}$ | $3.05 \cdot 10^7$ | 0.376 |
| 1 000 | $10^{-5}$ | $4.42 \cdot 10^7$ | 0.545 |
| 2 000 | $10^{-3}$ | $1.07 \cdot 10^7$ | 0.132 |
| 2 000 | $10^{-4}$ | $1.70 \cdot 10^7$ | 0.210 |
| 2 000 | $10^{-5}$ | $2.50 \cdot 10^7$ | 0.308 |

Table 1: Number of nonzeros of $A$ for different $\beta$ and $\varepsilon$.

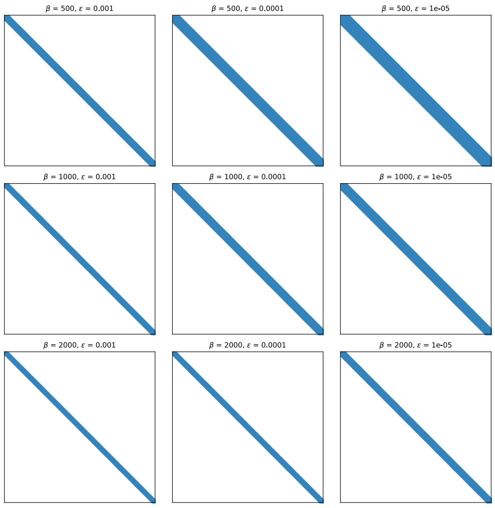

Figure 4: Sparsity pattern of $A$ for different $\beta$ (rows) and $\varepsilon$ (columns).

# 3 Results

## 3.1 Influence of the regularization parameter $\lambda$ on CG

We use the tolerance $10^{-8}$, at most 300 iterations, and $\lambda \in \{0, 0.01, 0.05, 0.3, 1\}$. The results are shown in Figure 5. With small $\lambda$, the relative residual does not shrink much, but the reconstruction error is smaller, except for $\lambda = 0$, where the error grows. For larger $\lambda$ the residual shrinks faster, but the error is larger.

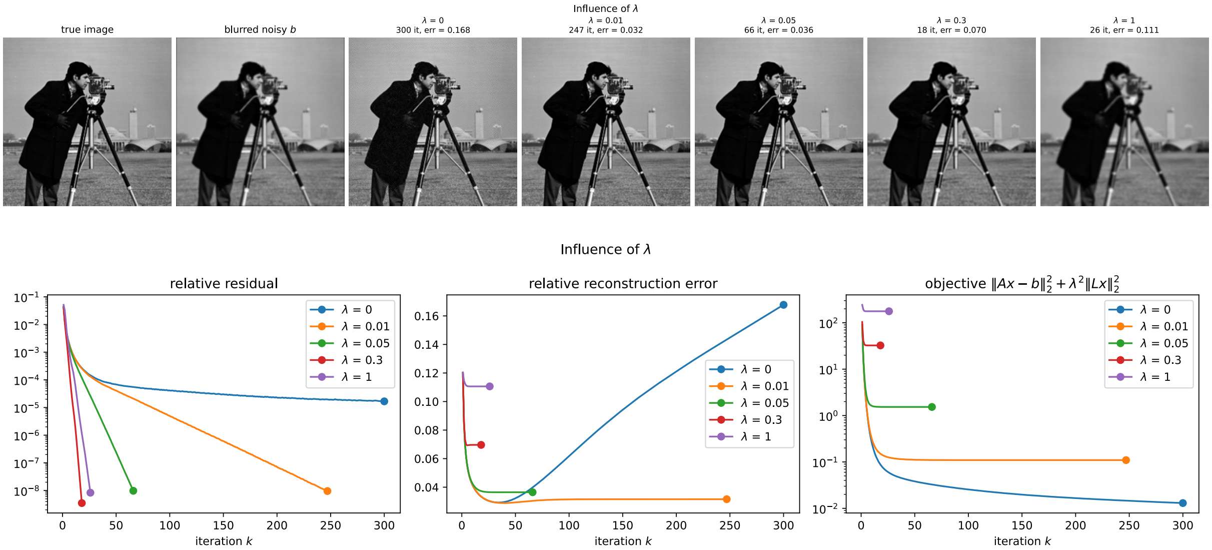

Figure 5: Influence of $\lambda$ on the CG reconstruction and convergence.

## 3.2 Influence of the stopping tolerance on CG

We fix $\lambda = 0.05$ and at most 300 iterations, and vary the tolerance. The results are shown in Figure 6. We see that the error and residual decrease with the tolerance, but the error levels off at for tolerances below 0.001. This means it is unnecessary to use a lower tolerance.

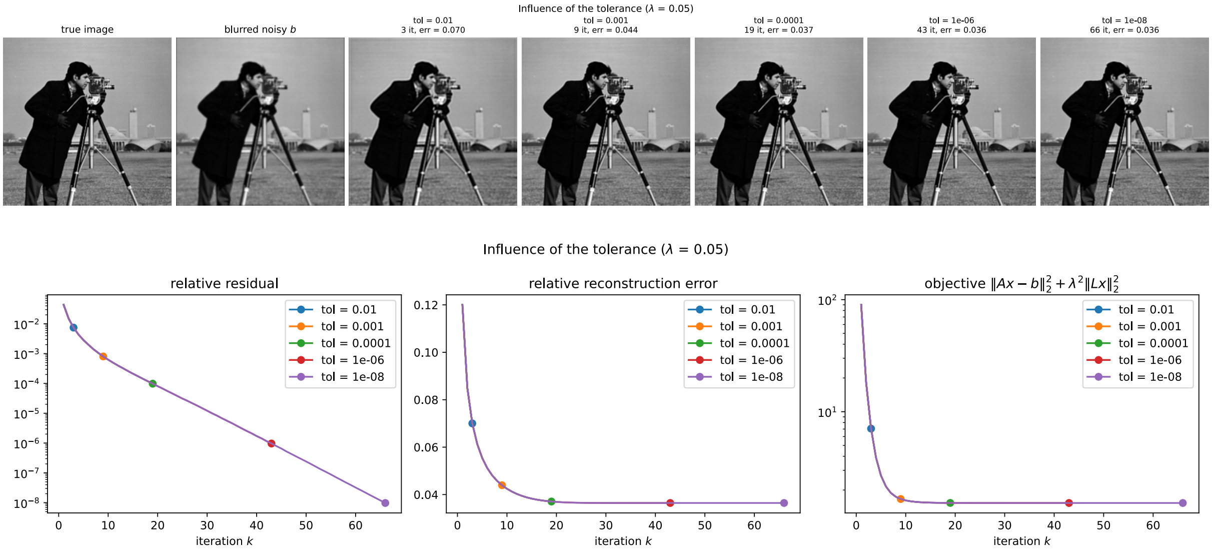

Figure 6: Influence of the tolerance on the CG reconstruction and convergence ($\lambda = 0.05$).

## 3.3 The maximum number of iterations for CG

We use a weak regularization, $\lambda = 0.01$, and tolerance 0, so CG always runs exactly `max_iter` iterations The results are shown in Figure 7. We see a similar pattern as for the tolerance: the error decreases with the number of iterations, but levels off for `max_iter` higher than 20.

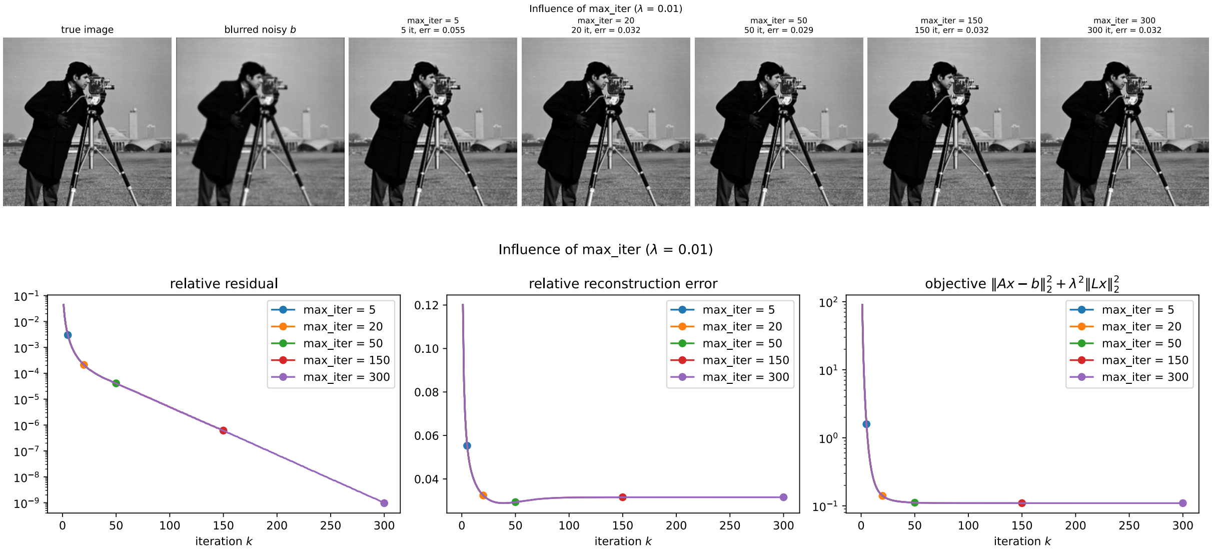

Figure 7: Influence of the maximum number of iterations on the CG reconstruction and convergence ($\lambda = 0.01$).

## 3.4 Comparison of CG and MINRES

CG and MINRES are compared in Figure 8 and Figure 9.

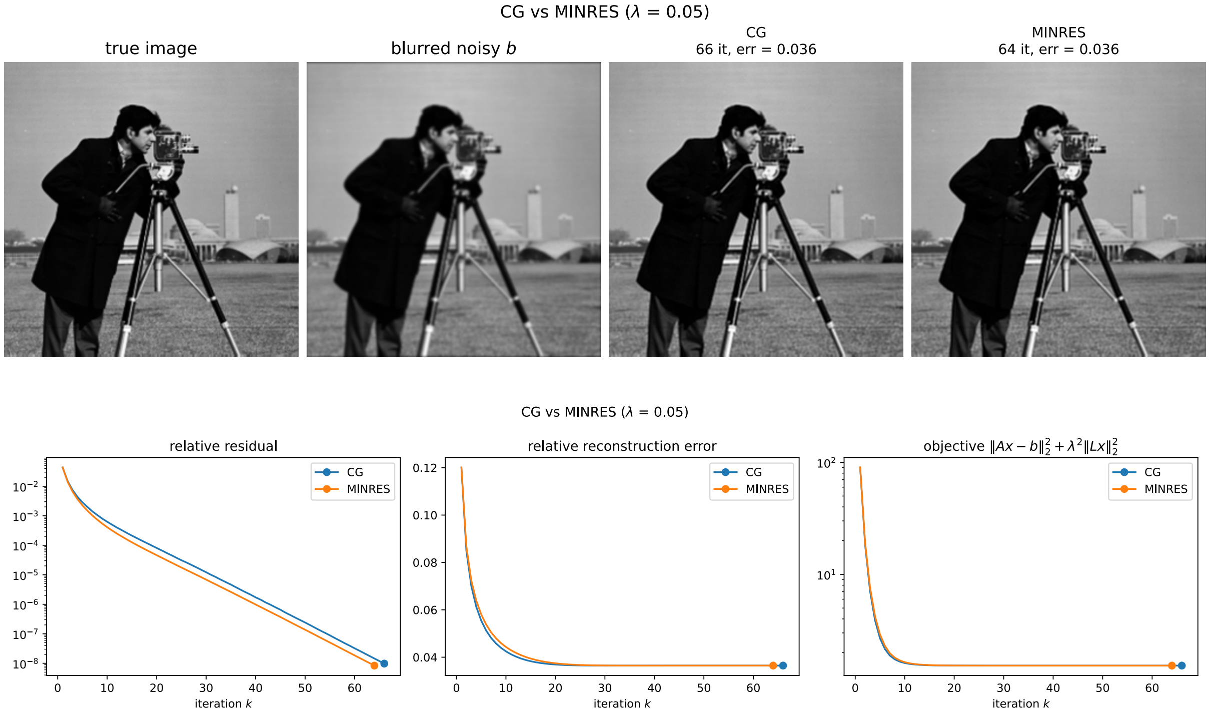

Figure 8: CG and MINRES for $\lambda = 0.05$, tolerance $10^{-8}$, at most 300 iterations.

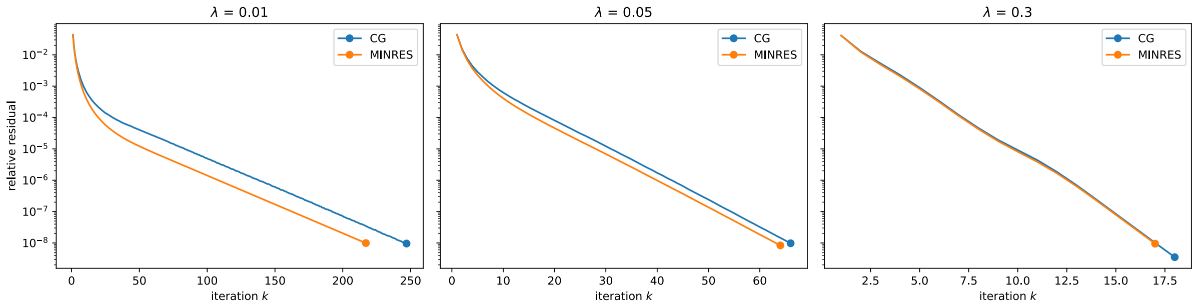

Figure 9: Relative residual of CG and MINRES for three values of $\lambda$.

| $\lambda$ | **Method** | **Iter.** | $Mv$ | **Sparse mat-vec** | **Time [s]** | **Rel. res.** | **Error** |
|---|---|---|---|---|---|---|---|
| 0.01 | CG | 247 | 248 | 992 | 2.68 | $9.6 \cdot 10^{-9}$ | 0.032 |
| | MINRES | 217 | 218 | 872 | 2.21 | $9.9 \cdot 10^{-9}$ | 0.032 |
| 0.05 | CG | 66 | 67 | 268 | 0.66 | $9.8 \cdot 10^{-9}$ | 0.036 |
| | MINRES | 64 | 65 | 260 | 0.7 | $8.3 \cdot 10^{-9}$ | 0.036 |
| 0.3 | CG | 18 | 19 | 76 | 0.18 | $3.5 \cdot 10^{-9}$ | 0.07 |
| | MINRES | 17 | 18 | 72 | 0.16 | $9.5 \cdot 10^{-9}$ | 0.07 |

Table 2: CG and MINRES with tolerance $10^{-8}$ and at most 300 iterations: iterations, products with $M$, sparse matrix-vector products, time, final relative residual and relative reconstruction error.

### 3.5 Why CG is expected to outperform MINRES

$M$ is symmetric positive definite, which is exactly the case CG is designed for. MINRES is designed for symmetric indefinite systems, which is a more general case. In every iteration CG gives the smallest possible value of the regularized least squares objective $f$ in the Krylov space. MINRES instead minimizes $\|c - Mx\|_2$. That makes CG a better fit here, and it usually needs fewer iterations and less work.

### 3.6 Preconditioned CG

The idea of preconditioning a problem is to turn it into a similar problem with a smaller condition number. To precondition for CG, the matrix has to stay SPD. A way of achieving this is with an incomplete Cholesky factorization $P = CC^T \approx M$, where $C$ is sparse and lower triangular, we get the SPD system

$$C^{-1}MC^{-T}\tilde{x} = C^{-1}c, \quad \tilde{x} = C^T x.$$

Applying CG to this system with $\widetilde{M} = C^{-1}MC^{-T}$ gives

$$\alpha_k = \frac{\tilde{r}_k^T \tilde{r}_k}{\tilde{p}_k^T \widetilde{M} \tilde{p}_k}, \quad \tilde{x}_{k+1} = \tilde{x}_k + \alpha_k \tilde{p}_k, \quad \tilde{r}_{k+1} = \tilde{r}_k - \alpha_k \widetilde{M} \tilde{p}_k,$$

$$\beta_k = \frac{\tilde{r}_{k+1}^T \tilde{r}_{k+1}}{\tilde{r}_k^T \tilde{r}_k}, \quad \tilde{p}_{k+1} = \tilde{r}_{k+1} + \beta_k \tilde{p}_k,$$

and the solution is recovered from $x = C^{-T}\tilde{x}$. In practice, $\widetilde{M}$ is never formed. With $r = C\tilde{r}$ and $p = C^{-T}\tilde{p}$, the same iteration can be carried out in the original variables, where every iteration only needs one product with $M$ and one solve with $P$ (two triangular solves).

We compare CG and PCG, both starting from $x_0 = 0$ and stopping when the relative residual is below $10^{-6}$ or after 500 iterations (Figure 10, Table 3).

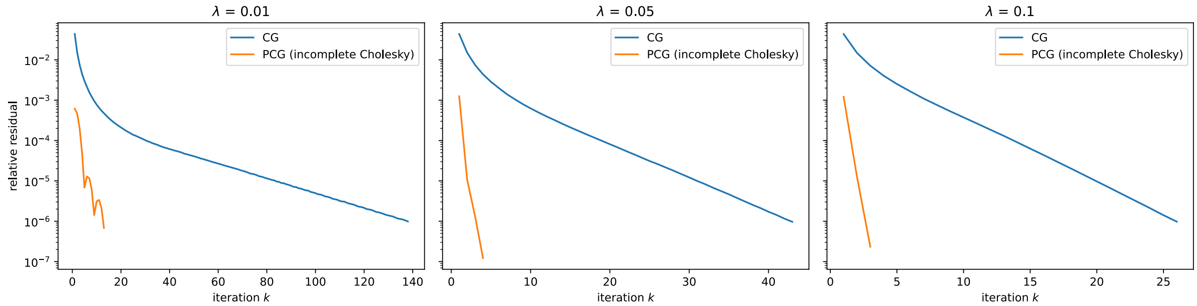

Figure 10: Relative residual of CG and PCG (incomplete Cholesky) for $\lambda = 0.01, 0.05, 0.1$.

| $\lambda$ | **Method** | **Iter.** | **Time [s]** | **Rel. res.** | **Error** |
|---|---|---|---|---|---|
| 0.01 | CG | 138 | 1.66 | $9.9 \cdot 10^{-7}$ | 0.0316 |
| | PCG | 13 | $1.99 + 15.6$ | $6.8 \cdot 10^{-7}$ | 0.0316 |
| 0.05 | CG | 43 | 0.58 | $9.7 \cdot 10^{-7}$ | 0.0365 |
| | PCG | 4 | $0.57 + 12.6$ | $1.2 \cdot 10^{-7}$ | 0.0365 |
| 0.1 | CG | 26 | 0.35 | $9.8 \cdot 10^{-7}$ | 0.0461 |
| | PCG | 3 | $0.47 + 12.7$ | $2.3 \cdot 10^{-7}$ | 0.0461 |

Table 3: CG and PCG with tolerance $10^{-6}$: iterations, time (for PCG: solve + building the incomplete Cholesky factor), final relative residual and relative reconstruction error.

Table 3 shows that PCG needs about 10 times fewer iterations than CG for all three values of $\lambda$. Despite this, PCG is not faster: the solve itself takes about as long as CG, since each iteration is much more expensive, and building the incomplete Cholesky factor adds time. Both methods reach the tolerance and give the same reconstruction error, since they solve the same system to the same tolerance.

# 4 Summary

For solving inverse problems, direct methods and iterative Krylov methods have significant differences in terms of computational complexity, memory requirements, numerical stability, scalability, and suitability for small and large problems. Direct methods require significant memory and computational resources, using dense matrix operations, especially for large problems. However, they are often more numerically stable and can provide better solutions. Krylov subspace methods are more memory-efficient and scalable, making them suitable for large problems, but they may be less stable and require careful tuning of parameters. Direct methods are advantageous for small problems due to their accuracy, while iterative methods are preferred in large-scale applications.

# References

[1] D. Mirzaei, “Iterative Methods in Matrix Computations.” 2026.

[2] C. C. Paige and M. A. Saunders, “Algorithm 583: LSQR: Sparse Linear Equations and Least Squares Problems,” *ACM Transactions on Mathematical Software*, vol. 8, no. 2, pp. 195–209, 1982, [Online]. Available: https://web.stanford.edu/group/SOL/software/lsqr/lsqr-toms82b.pdf

[3] “What does LSQR stand for?.” [Online]. Available: https://math.stackexchange.com/questions/4075853/what-does-lsqr-stand-for